

Project 1 - xv6 OS Assignment

Group members: Ankrit Gupta (solo submission)

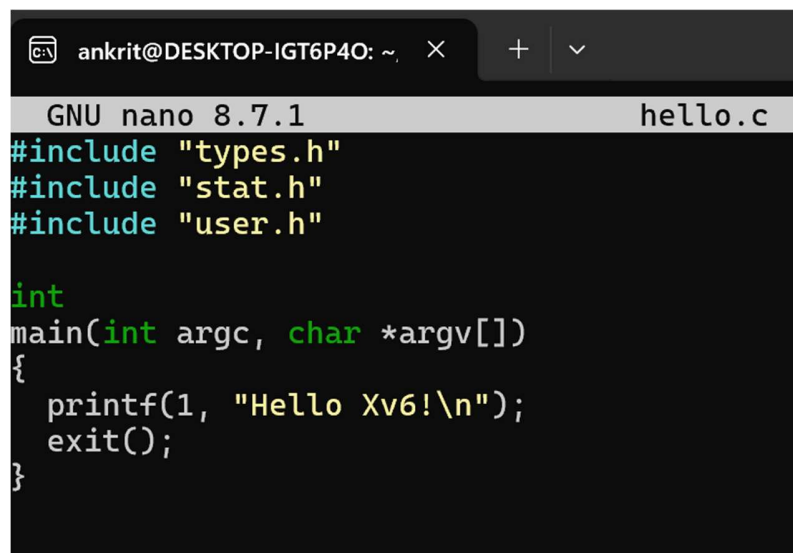
Date: September 28, 2026

Environment

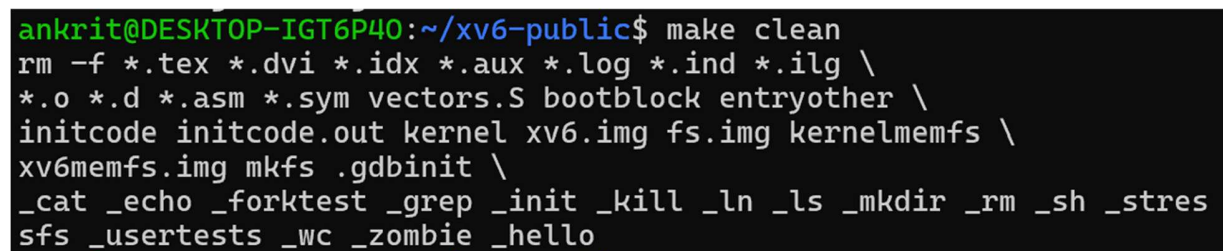
- OS: Windows Subsystem for Linux 2 (WSL2), Ubuntu
 - Kernel: Linux DESKTOP-IGT6P4O 6.18.33.2-microsoft-standard-WSL2
- Compiler: gcc (Ubuntu 15.2.0-16ubuntu1) 15.2.0
- Emulator: QEMU emulator version 10.2.1 (Debian 1:10.2.1+ds-1ubuntu3.2)
- xv6 source: github.com/mit-pdos/xv6-public (x86)
- Built with: make
- Run with: make qemu-nox

Part 1

The hello.c program



```
ankrit@DESKTOP-IGT6P4O: ~  
GNU nano 8.7.1 hello.c  
#include "types.h"  
#include "stat.h"  
#include "user.h"  
  
int  
main(int argc, char *argv[])  
{  
    printf(1, "Hello Xv6!\n");  
    exit();  
}
```



```
ankrit@DESKTOP-IGT6P4O:~/xv6-public$ make clean  
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \  
*.o *.d *.asm *.sym vectors.S bootblock entryother \  
initcode initcode.out kernel xv6.img fs.img kernelmemfs \  
xv6memfs.img mkfs .gdbinit \  
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stres  
sfs _usertests _wc _zombie _hello
```

Make command finished successfully.

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
-ggdb -m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-i
nfinite-recursion -fno-stack-protector -fno-pie -no-pie -fno-pic -O
-nostdinc -I. -c bootmain.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
-ggdb -m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-i
nfinite-recursion -fno-stack-protector -fno-pie -no-pie -fno-pic -no
stdinc -I. -c bootasm.S
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o
bootmain.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblock.o has a LOAD se
gment with RWX permissions
```

```
-b binary initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kern
el.sym
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.0141053 s, 366 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000348814 s, 512 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
395+1 records in
395+1 records out
202392 bytes (202 kB, 198 KiB) copied, 0.000989523 s, 202 MB/s
ankrit@DESKTOP-IGT6P40:~/xv6-public$
```

Hello in action

```
$  
$  
$  
$ hello  
Hello Xv6!  
$  
$  
$  
$ hello  
Hello Xv6!  
$  
$  
$
```

Part 2

Output of ls command before modification

```
ankrit@DESKTOP-IGT6P4O: ~  
SeaBIOS (version 1.17.0-debian-1.17.0-1ubu  
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.  
  
Booting from Hard Disk..xv6...  
cpu0: starting 0  
sb: size 1000 nblocks 941 ninodes 200 nlog  
init: starting sh  
$ ls  
.  
..  
README  
cat  
echo  
forktest  
grep  
init  
kill  
ln  
ls  
mkdir  
rm  
sh  
stressfs  
usertests  
wc  
zombie  
hello  
sleep  
console  
$
```

Updated ls.c program.

```
ankrit@DESKTOP-IGT6P4O: ~ × + ▾
GNU nano 8.7.1 ls.c
#include "types.h"
#include "stat.h"
#include "user.h"
#include "fs.h"

int showall = 0; // set to 1 when -a is given

char*
fmtname(char *path, int isdir)
{
    static char buf[DIRSIZ+2]; // +1 for the '/', +1 for the termin>
    char *p;
    int len;

    // Find first character after last slash.
    for(p=path+strlen(path); p >= path && *p != '/'; p--)
        ;
    p++;

    len = strlen(p);
    if(len > DIRSIZ)
        len = DIRSIZ;
    memmove(buf, p, len);
    if(isdir)
        buf[len++] = '/'; // slash goes right after the name...
    while(len < DIRSIZ)
        buf[len++] = ' '; // ...then pad with spaces
    buf[len] = 0;
    return buf;
}

void
ls(char *path)
{
    char buf[512], *p;
    int fd;
    struct dirent de;
    struct stat st;
```

```
ankrit@DESKTOP-IGT6P4O: ~  
if((fd = open(path, 0)) < 0){  
    printf(2, "ls: cannot open %s\n", path);  
    return;  
}  
  
if(fstat(fd, &st) < 0){  
    printf(2, "ls: cannot stat %s\n", path);  
    close(fd);  
    return;  
}  
  
switch(st.type){  
case T_FILE:  
    printf(1, "%s %d %d %d\n", fmtname(path, 0), st.type, st.ino, st.  
.size);  
    break;  
  
case T_DIR:  
    if(strlen(path) + 1 + DIRSIZ + 1 > sizeof buf){  
        printf(1, "ls: path too long\n");  
        break;  
    }  
    strcpy(buf, path);  
    p = buf+strlen(buf);  
    *p++ = '/';  
    while(read(fd, &de, sizeof(de)) == sizeof(de)){  
        if(de.inum == 0)  
            continue;  
        if(!showall && de.name[0] == '.') // hide dotfiles unless -a  
            continue;  
        memmove(p, de.name, DIRSIZ);  
        p[DIRSIZ] = 0;  
        if(stat(buf, &st) < 0){  
            printf(1, "ls: cannot stat %s\n", buf);  
            continue;  
        }  
        printf(1, "%s %d %d %d\n", fmtname(buf, st.type == T_DIR), st.  
type, st.ino, st.size);  
    }  
    break;  
}  
close(fd);
```

```

    close(fd);
}

int
main(int argc, char *argv[])
{
    int i, npaths = 0;

    // Pass 1: find the flag, so "ls -a dir" and "ls dir -a" both work
    for(i = 1; i < argc; i++){
        if(strcmp(argv[i], "-a") == 0)
            showall = 1;
    }
    // Pass 2: list every argument that isn't the flag.
    for(i = 1; i < argc; i++){
        if(strcmp(argv[i], "-a") == 0)
            continue;
        ls(argv[i]);
        npaths++;
    }
    if(npaths == 0)        // no path given: list the current directory
        ls(".");
    exit();
}

```

Git difference of ls.c

```

ankrit@DESKTOP-IGT6P40:~/xv6-public$ git diff main -- ls.c
diff --git a/ls.c b/ls.c
index 2862913..6dff969 100644
--- a/ls.c
+++ b/ls.c
@@ -3,22 +3,29 @@
#include "user.h"
#include "fs.h"

+int showall = 0;    // set to 1 when -a is given
+
char*
-fmtname(char *path)
+fmtname(char *path, int isdir)
{
- static char buf[DIRSIZ+1];
+ static char buf[DIRSIZ+2];    // +1 for the '/', +1 for the terminator
  char *p;
+ int len;

  // Find first character after last slash.
  for(p=path+strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

- // Return blank-padded name.
- if(strlen(p) >= DIRSIZ)
-   return p;
- memmove(buf, p, strlen(p));
- memset(buf+strlen(p), ' ', DIRSIZ-strlen(p));
+ len = strlen(p);
+ if(len > DIRSIZ)
+   len = DIRSIZ;
+ memmove(buf, p, len);
+ if(isdir)
+   buf[len++] = '/';           // slash goes right after the name...
+ while(len < DIRSIZ)
+   buf[len++] = ' ';          // ...then pad with spaces
+ buf[len] = 0;
  return buf;
}

```

```

ankrit@DESKTOP-IGT6P4O: ~
return buf;
}

@@ -43,7 +50,7 @@ ls(char *path)

switch(st.type){
case T_FILE:
-   printf(1, "%s %d %d %d\n", fmtname(path), st.type, st.ino, st.size);
+   printf(1, "%s %d %d %d\n", fmtname(path, 0), st.type, st.ino, st.size);
+   break;

case T_DIR:
@@ -57,13 +64,15 @@ ls(char *path)
while(read(fd, &de, sizeof(de)) == sizeof(de)){
    if(de.inum == 0)
        continue;
+   if(!showall && de.name[0] == '.') // hide dotfiles unless -a
+   continue;
    memmove(p, de.name, DIRSIZ);
    p[DIRSIZ] = 0;
    if(stat(buf, &st) < 0){
        printf(1, "ls: cannot stat %s\n", buf);
        continue;
    }
-   printf(1, "%s %d %d %d\n", fmtname(buf), st.type, st.ino, st.size);
+   printf(1, "%s %d %d %d\n", fmtname(buf), st.type, st.ino, st.size);
+   break;
}
}

@@ -73,13 +82,21 @@ ls(char *path)
int
main(int argc, char *argv[])
{
-   int i;
+   int i, npaths = 0;

-   if(argc < 2){
-       ls(".");
-       exit();
+   // Pass 1: find the flag, so "ls -a dir" and "ls dir -a" both work.
+   for(i = 1; i < argc; i++){
+       if(strcmp(argv[i], "-a") == 0)
+           showall = 1;
+   }
-   for(i=1; i<argc; i++)
+   // Pass 2: list every argument that isn't the flag.
+   for(i = 1; i < argc; i++){
+       if(strcmp(argv[i], "-a") == 0)
+           continue;
+       ls(argv[i]);
+       npaths++;
+   }
+   if(npaths == 0) // no path given: list the current directory
+   ls(".");
+   exit();
}
(END)

```

(END)

```
case T_FILE:
-   printf(1, "%s %d %d %d\n", fmtname(path), st.type, st.ino, st.size);
+   printf(1, "%s %d %d %d\n", fmtname(path, 0), st.type, st.ino, st.size);
    break;

case T_DIR:
@@ -57,13 +64,15 @@ ls(char *path)
    while(read(fd, &de, sizeof(de)) == sizeof(de)){
        if(de.inum == 0)
            continue;
+       if(!showall && de.name[0] == '.')    // hide dotfiles unless -a
+           continue;
        memmove(p, de.name, DIRSIZ);
        p[DIRSIZ] = 0;
        if(stat(buf, &st) < 0){
            printf(1, "ls: cannot stat %s\n", buf);
            continue;
        }
-       printf(1, "%s %d %d %d\n", fmtname(buf), st.type, st.ino, st.size);
+       printf(1, "%s %d %d %d\n", fmtname(buf, st.type == T_DIR), st.type, st.ino, st.size);
    }
    break;
}
@@ -73,13 +82,21 @@ ls(char *path)
int
main(int argc, char *argv[])
{
-   int i;
+   int i, npaths = 0;

-   if(argc < 2){
-       ls(".");
-       exit();
+   // Pass 1: find the flag, so "ls -a dir" and "ls dir -a" both work.
+   for(i = 1; i < argc; i++){
+       if(strcmp(argv[i], "-a") == 0)
+           showall = 1;
+   }
-   for(i=1; i<argc; i++)
+   // Pass 2: list every argument that isn't the flag.
+   for(i = 1; i < argc; i++){
+       if(strcmp(argv[i], "-a") == 0)
+           continue;
+       ls(argv[i]);
+       npaths++;
+   }
+   if(npaths == 0)        // no path given: list the current directory
+       ls(".");
    exit();
}
~
~
~
~
```

Changes after make clean and make.

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stre
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MS
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblock.o has a LOAD ss
objdump -S bootblock.o > bootblock.asm
objcopy -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
boot block is 453 bytes (max 510)
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -m32 -gdwarf-2 -Wa,-divide -c -o swtch.o swtch.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -m32 -gdwarf-2 -Wa,-divide -c -o trapasm.o trapasm.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
./vectors.pl > vectors.S
gcc -m32 -gdwarf-2 -Wa,-divide -c -o vectors.o vectors.S
```

```

./vectors.pl > vectors.S
gcc -m32 -gdwarf-2 -Wa,-divide -c -o vectors.o vectors.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -Mc
gcc -m32 -gdwarf-2 -Wa,-divide -c -o entry.o entry.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MS
ld -m elf_i386 -N -e start -Ttext 0x7000 -o bootblockother.o ento
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblockother.o has a Ls
objcopy -S -O binary -j .text bootblockother.o entryother
objdump -S bootblockother.o > entryother.asm
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MS
ld -m elf_i386 -N -e start -Ttext 0 -o initcode.out initcode.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: initcode.out has a LOAD s
objcopy -S -O binary initcode.out initcode
objdump -S initcode.o > initcode.asm
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exr
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kerm
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.0133442 s, 394 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000276161 s, 512 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
395+1 records in
395+1 records out
202392 bytes (202 kB, 198 KiB) copied, 0.000981504 s, 202 MB/s
ankrit@DESKTOP-IGT6P40:~/xv6-public$

```

After creating the files and ls screenshot.

Ls output after the modification

```
ankrit@DESKTOP-IGT6P40: ~  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inod  
init: starting sh  
$ ls  
README          2 2 2286  
cat              2 3 16060  
echo            2 4 14952  
forktest        2 5 9008  
grep            2 6 18912  
init            2 7 15560  
kill            2 8 15028  
ln              2 9 14896  
ls              2 10 18080  
mkdir           2 11 15024  
rm              2 12 15008  
sh              2 13 29016  
stressfs        2 14 15852  
usertests       2 15 63736  
wc              2 16 16360  
zombie          2 17 14596  
hello           2 18 14768  
sleep           2 19 14888  
console         3 20 0  
$  
$  
$ mkdir docs  
$ mkdir .secret  
$ echo hi > .secret  
$ echo hi > visible.txt  
$  
$ ls  
README          2 2 2286  
cat              2 3 16060  
echo            2 4 14952  
forktest        2 5 9008  
grep            2 6 18912  
init            2 7 15560  
kill            2 8 15028  
ln              2 9 14896  
ls              2 10 18080  
mkdir           2 11 15024  
rm              2 12 15008  
sh              2 13 29016  
stressfs        2 14 15852  
usertests       2 15 63736  
wc              2 16 16360  
zombie          2 17 14596  
hello           2 18 14768  
sleep           2 19 14888  
console         3 20 0  
docs/           1 21 32  
visible.txt     2 24 3  
$ |
```

```
$  
$ ls -a  
./          1 1 512  
../         1 1 512  
README     2 2 2286  
cat        2 3 16036  
echo       2 4 14932  
forktest   2 5 8976  
grep       2 6 18888  
init       2 7 15536  
kill       2 8 15008  
ln         2 9 14872  
ls         2 10 18056  
mkdir      2 11 15004  
rm         2 12 14984  
sh         2 13 28992  
stressfs   2 14 15832  
usertests  2 15 63712  
wc         2 16 16340  
zombie     2 17 14572  
hello      2 18 14700  
sleep      2 19 14868  
console    3 20 0  
docs/      1 21 32  
.secretdir/ 1 22 32  
.secret     2 23 3  
visible.txt 2 24 3  
$  
$ ls docs  
$  
$  
$  
$ ls -a docs  
./          1 21 32  
../         1 1 512  
$  
$  
$ |
```

Part 3:

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$  
ankrit@DESKTOP-IGT6P40:~/xv6-public$ grep -n getpid *.c *.h *.S  
syscall.c:92:extern int sys_getpid(void);  
syscall.c:119:[SYS_getpid] sys_getpid,  
sysproc.c:40:sys_getpid(void)  
usertests.c:434: ppid = getpid();  
usertests.c:1498: ppid = getpid();  
syscall.h:12:#define SYS_getpid 11  
user.h:22:int getpid(void);  
usys.S:28:SYSCALL(getpid)  
ankrit@DESKTOP-IGT6P40:~/xv6-public$ |
```

Files containing getpid: syscall.c, syscall.h, sysproc.c, user.h, usys.S (usertests.c also calls getpid but was not modified, it only tests it). These same five files were edited to add the hello() syscall.

Git Diff files:

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ git diff
diff --git a/hello.c b/hello.c
index 59bbc66..3e4fd0d 100644
--- a/hello.c
+++ b/hello.c
@@ -6,5 +6,6 @@ int
main(int argc, char *argv[])
{
    printf(1, "Hello Xv6!\n");
+   hello();
    exit();
}
diff --git a/syscall.c b/syscall.c
index ee85261..1e5198e 100644
--- a/syscall.c
+++ b/syscall.c
@@ -103,6 +103,7 @@ extern int sys_unlink(void);
extern int sys_wait(void);
extern int sys_write(void);
extern int sys_uptime(void);
+extern int sys_hello(void);

static int (*syscalls[])(void) = {
[SYS_fork]    sys_fork,
@@ -126,6 +127,7 @@ static int (*syscalls[])(void) = {
[SYS_link]    sys_link,
[SYS_mkdir]   sys_mkdir,
[SYS_close]   sys_close,
+[SYS_hello]   sys_hello,
};

void
```

```

diff --git a/syscall.h b/syscall.h
index bc5f356..5f48d9b 100644
--- a/syscall.h
+++ b/syscall.h
@@ -20,3 +20,4 @@
 #define SYS_link    19
 #define SYS_mkdir   20
 #define SYS_close   21
+#define SYS_hello   22
diff --git a/sysproc.c b/sysproc.c
index 0686d29..0c1a098 100644
--- a/sysproc.c
+++ b/sysproc.c
@@ -89,3 +89,12 @@ sys_uptime(void)
     release(&tickslock);
     return xticks;
 }
+
+
+//return hello from the kernel for our custom hello message
+int
+sys_hello(void)
+{
+    cprintf("Hello from the kernel!\n");
+    return 0;
+}
diff --git a/user.h b/user.h
index 4f99c52..2f95ca5 100644
--- a/user.h
+++ b/user.h
@@ -23,6 +23,7 @@ int getpid(void);
 char* sbrk(int);
 int sleep(int);
 int uptime(void);
+int hello(void);

// ulib.c
int stat(const char*, struct stat*);
diff --git a/usys.S b/usys.S
index 8bfd8a1..0072273 100644
--- a/usys.S
+++ b/usys.S
@@ -29,3 +29,4 @@ SYSCALL(getpid)
 SYSCALL(sbrk)
 SYSCALL(sleep)
 SYSCALL(uptime)
+SYSCALL(hello)

```

Make clean and make command after file changes:

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stressfs _use
rttests _wc _zombie _hello _sleep
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -
m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-infinite-recursio
n -fno-stack-protector -fno-pie -no-pie -fno-pic -O -nostdinc -I. -c bootmai
n.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -
m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-infinite-recursio
n -fno-stack-protector -fno-pie -no-pie -fno-pic -nostdinc -I. -c bootasm.S
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o bootmai
n.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblock.o has a LOAD segment wi
th RWX permissions
objdump -S bootblock.o > bootblock.asm
objcopy -S -O binary -j .text bootblock.o bootblock
./sign.pl bootblock
boot block is 453 bytes (max 510)
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -
m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-infinite-recursio
n -fno-stack-protector -fno-pie -no-pie -c -o bio.o bio.c
```

```

gcc -m32 -gdwarf-2 -Wa,-divide -c -o vectors.o vectors.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb c
gcc -m32 -gdwarf-2 -Wa,-divide -c -o entry.o entry.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb S
ld -m elf_i386 -N -e start -Ttext 0x7000 -o bootblockother.o entryother.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblockother.o has a LOAD segms
objcopy -S -O binary -j .text bootblockother.o entryother
objdump -S bootblockother.o > entryother.asm
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb S
ld -m elf_i386 -N -e start -Ttext 0 -o initcode.out initcode.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: initcode.out has a LOAD segment s
objcopy -S -O binary initcode.out initcode
objdump -S initcode.o > initcode.asm
ld -m elf_i386 -T kernel.ld -o kernel entry.o bio.o console.o exec.o filr
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kernel.sym
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.0132997 s, 394 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000273455 s, 512 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
395+1 records in
395+1 records out
202516 bytes (203 kB, 198 KiB) copied, 0.000929159 s, 203 MB/s
ankrit@DESKTOP-IGT6P40:~/xv6-public$ |

```

Output of the hello syscall.

```

ankrit@DESKTOP-IGT6P40: ~
SeaBIOS (version 1.17.0-debian-1.17.0-1ubuntu1)

iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC8E00+1EF08E00 CA00

Booting from Hard Disk..xv6...
cpu0: starting 0
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap8
init: starting sh
$ hello
Hello Xv6!
Hello from the kernel!
$ |

```

Part 4

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ grep -n -A15 "sys_sleep" sysproc.c
60:sys_sleep(void)
61-{
62-    int n;
63-    uint ticks0;
64-
65-    if(argint(0, &n) < 0)
66-        return -1;
67-    acquire(&tickslock);
68-    ticks0 = ticks;
69-    while(ticks - ticks0 < n){
70-        if(myproc()->killed){
71-            release(&tickslock);
72-            return -1;
73-        }
74-        sleep(&ticks, &tickslock);
75-    }
```

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ cat sleep.c
#include "types.h"
#include "stat.h"
#include "user.h"

int
main(int argc, char *argv[])
{
    if(argc < 2){
        printf(2, "Usage: sleep <ticks>\n");
        exit();
    }
    sleep(atoi(argv[1]));
    exit();
}
```

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stres
sfs _usertests _wc _zombie _hello _sleep
```

Make command finished successfully.

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
-ggdb -m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-i
nfinite-recursion -fno-stack-protector -fno-pie -no-pie -fno-pic -O
-nostdinc -I. -c bootmain.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD
-ggdb -m32 -Werror -fno-omit-frame-pointer -Wno-array-bounds -Wno-i
nfinite-recursion -fno-stack-protector -fno-pie -no-pie -fno-pic -no
stdinc -I. -c bootasm.S
ld -m elf_i386 -N -e start -Ttext 0x7C00 -o bootblock.o bootasm.o
bootmain.o
/usr/bin/x86_64-linux-gnu-ld.bfd: warning: bootblock.o has a LOAD se
gment with RWX permissions
```

```
-b binary initcode entryother
objdump -S kernel > kernel.asm
objdump -t kernel | sed '1,/SYMBOL TABLE/d; s/ .* / /; /^$/d' > kern
el.sym
dd if=/dev/zero of=xv6.img count=10000
10000+0 records in
10000+0 records out
5120000 bytes (5.1 MB, 4.9 MiB) copied, 0.0141053 s, 366 MB/s
dd if=bootblock of=xv6.img conv=notrunc
1+0 records in
1+0 records out
512 bytes copied, 0.000348814 s, 512 kB/s
dd if=kernel of=xv6.img seek=1 conv=notrunc
395+1 records in
395+1 records out
202392 bytes (202 kB, 198 KiB) copied, 0.000989523 s, 202 MB/s
ankrit@DESKTOP-IGT6P40:~/xv6-public$
```

Sleep in action

```
$  
$  
$  
$ sleep  
Usage: sleep <ticks>  
$  
$  
$ sleep 100  
$  
$  
$ sleep 500  
$  
$  
$
```

Makefile changes: added `_hello` and `_sleep` to `UPROGS` so both programs are compiled and copied onto the `xv6` disk image, and added `hello.c` `sleep.c` to `EXTRA` for packaging.

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ grep -n -A 20 "UPROGS=" Makefile  
169:UPROGS=\  
170-   _cat\  
171-   _echo\  
172-   _forktest\  
173-   _grep\  
174-   _init\  
175-   _kill\  
176-   _ln\  
177-   _ls\  
178-   _mkdir\  
179-   _rm\  
180-   _sh\  
181-   _stressfs\  
182-   _usertests\  
183-   _wc\  
184-   _zombie\  
185-   _hello\  
186-   _sleep\  
187-  
188-fs.img: mkfs README $(UPROGS)  
189-   ./mkfs fs.img README $(UPROGS)  
ankrit@DESKTOP-IGT6P40:~/xv6-public$ ^C  
ankrit@DESKTOP-IGT6P40:~/xv6-public$ grep -n "hello.c sleep.c" Makefile  
256:   printf.c umalloc.c hello.c sleep.c\  
ankrit@DESKTOP-IGT6P40:~/xv6-public$
```

Final run check:

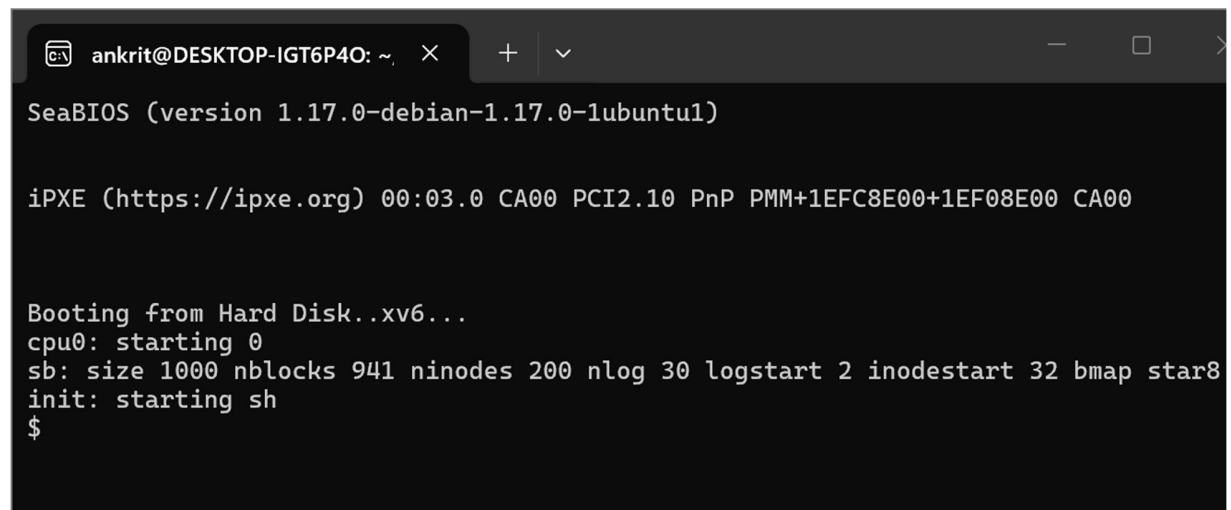
Output of make clean

```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make clean
rm -f *.tex *.dvi *.idx *.aux *.log *.ind *.ilg \
*.o *.d *.asm *.sym vectors.S bootblock entryother \
initcode initcode.out kernel xv6.img fs.img kernelmemfs \
xv6memfs.img mkfs .gdbinit \
_cat _echo _forktest _grep _init _kill _ln _ls _mkdir _rm _sh _stressfs _us
ankrit@DESKTOP-IGT6P40:~/xv6-public$
```

Final Make output:


```
ankrit@DESKTOP-IGT6P40:~/xv6-public$ make qemu-nox
gcc -Werror -Wall -o mkfs mkfs.c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 c
gcc -m32 -gdwarf-2 -Wa,-divide -c -o usys.o usys.S
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 c
gcc -fno-pic -static -fno-builtin -fno-strict-aliasing -O2 -Wall -MD -ggdb -m32 c
```

Booting into xv-6



```
ankrit@DESKTOP-IGT6P40: ~  
SeaBIOS (version 1.17.0-debian-1.17.0-1ubuntu1)  
  
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC8E00+1EF08E00 CA00  
  
Booting from Hard Disk..xv6...  
cpu0: starting 0  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star8  
init: starting sh  
$
```

Final program outputs.

```
ankrit@DESKTOP-IGT6P4O: ~  
SeaBIOS (version 1.17.0-debian-1.17.0-1ubuntu1)  
  
iPXE (https://ipxe.org) 00:03.0 CA00 PCI2.10 PnP PMM+1EFC8E00+1EF08E00 CA00  
  
Booting from Hard Disk..xv6...  
cpu0: starting 0  
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inodestart 32 bmap star8  
init: starting sh  
$  
$  
$  
$ hello  
Hello Xv6!  
Hello from the kernel!  
$  
$  
$ sleep  
Usage: sleep <ticks>  
$  
$  
$ sleep 100  
$  
$  
$ sleep 500  
$  
$  
$ |
```

ankrit@DESKTOP-IGT6P4O: ~

```
sb: size 1000 nblocks 941 ninodes 200 nlog 30 logstart 2 inod
init: starting sh
$ ls
README          2  2  2286
cat              2  3 16060
echo            2  4 14952
forktest        2  5  9008
grep            2  6 18912
init            2  7 15560
kill            2  8 15028
ln              2  9 14896
ls              2 10 18080
mkdir           2 11 15024
rm              2 12 15008
sh              2 13 29016
stressfs        2 14 15852
usertests       2 15 63736
wc              2 16 16360
zombie          2 17 14596
hello           2 18 14768
sleep           2 19 14888
console         3 20  0
$
$
$
$ mkdir docs
$ mkdir .secretdir
$ echo hi > .secret
$ echo hi > visible.txt
$
$ ls
README          2  2  2286
cat              2  3 16060
echo            2  4 14952
forktest        2  5  9008
grep            2  6 18912
init            2  7 15560
kill            2  8 15028
ln              2  9 14896
ls              2 10 18080
mkdir           2 11 15024
rm              2 12 15008
sh              2 13 29016
stressfs        2 14 15852
usertests       2 15 63736
wc              2 16 16360
zombie          2 17 14596
hello           2 18 14768
sleep           2 19 14888
console         3 20  0
docs/           1 21  32
visible.txt     2 24  3
$ |
```

```
$  
$ ls -la  
./          1 1 512  
../         1 1 512  
README     2 2 2286  
cat        2 3 16036  
echo       2 4 14932  
forktest   2 5 8976  
grep       2 6 18888  
init       2 7 15536  
kill       2 8 15008  
ln         2 9 14872  
ls         2 10 18056  
mkdir      2 11 15004  
rm         2 12 14984  
sh         2 13 28992  
stressfs   2 14 15832  
usertests  2 15 63712  
wc         2 16 16340  
zombie     2 17 14572  
hello      2 18 14700  
sleep      2 19 14868  
console    3 20 0  
docs/      1 21 32  
.secretdir/ 1 22 32  
.secret     2 23 3  
visible.txt 2 24 3  
$  
$ ls docs  
$  
$  
$  
$ ls -la docs  
./          1 21 32  
../         1 1 512  
$  
$  
$ |
```

Resources used

- Claude (Anthropic AI assistant): Used to understand xv6's system program architecture - how user programs link against xv6's own C library instead of standard libc, how a syscall travels from a user program through `usys.S` and `syscall.c` into the kernel, and how the Makefile's `UPROGS/EXTRA` variables package and link user programs onto the xv6 disk image.

- xv6 source files (`echo.c`, `getpid/sys_getpid`, `sysproc.c`, `ls.c`): Read as reference for argv handling, kernel printing conventions (`cprintf` vs `printf`), and existing syscall/command patterns before writing my own implementations.